

Guide d'Administration et de Maintenance — La Termitière

Version : 1.0
Statut : Production
Cible : Équipe technique / DevOps
Dernière mise à jour : Juillet 2026

1. Objectif

Ce document centralise toutes les commandes, astuces et procédures nécessaires pour maintenir, administrer et déboguer l'infrastructure de La Termitière au quotidien, sans casser la production.

2. Exécution de Scripts Node.js (Migrations, Imports)

Dans l'idéal DevOps pur, on n'installe pas Node.js sur le serveur. Cependant, pour simplifier tes opérations de migration complexes, nous avons explicitement installé Node.js 20 sur ton VPS (cf. Guide d'Installation).

Tu peux donc lancer tes scripts d'administration nativement, sans t'encombrer de Docker.

2.1 Lancement natif (Direct sur le VPS)

Si tu dois exécuter `import.mjs` ou `auth-migrate.mjs` :

```
cd /home/bawa/termitiere-platform/migration/supabase

# Installer les dépendances si ce n'est pas déjà fait
npm install

# Exporter les variables nécessaires
export DATABASE_URL='postgresql://postgres:T0N_POSTGRES_PASSWORD@localhost:5432/postgres'
export SUPABASE_URL='https://api.laternitiere.com'
export SUPABASE_SERVICE_KEY='TA_SERVICE_ROLE_KEY'

# Lancer le script (avec le bypass SSL si HTTPS n'est pas encore actif localement)
NODE_TLS_REJECT_UNAUTHORIZED=0 node import.mjs
```

2.2 Alternative avancée : Le conteneur éphémère

Si un jour tu veux exécuter un script sans utiliser le Node.js installé sur ta machine (pour éviter les conflits de version), tu peux utiliser Docker :

```
cd /home/bawa/termitiere-platform/migration/supabase

# Lancer un conteneur temporaire
docker run -it --rm \
  --network supabase_default \
  -v /home/bawa/termitiere-platform:/app \
  -w /app/migration/supabase \
  node:20 bash
```

(Le `--rm` garantit que le conteneur se détruit tout seul dès que tu as fini).

Une fois à l'intérieur du conteneur (ton terminal affichera `root@xxxx:/app/migration/supabase#`), tu dois cibler la base de données via le réseau Docker interne (l'hôte devient `db` au lieu de `localhost`) :

```
# 1. Installer les dépendances
npm install

# 2. Configurer la variable d'environnement
export DATABASE_URL='postgresql://postgres:T0N_POSTGRES_PASSWORD@db:5432/postgres'

# 3. Lancer le script
node import.mjs
```

3. Commandes de Survie Docker (Le Quotidien)

3.1 Lire les logs en temps réel (Très utile)

Pour voir ce qu'il se passe sur un service précis (ex: le frontend web ou un service Supabase) :

```
# Voir les logs du Frontend
docker logs -f termitiere-web

# Voir les logs de l'API Supabase
docker logs -f supabase-kong

# Voir les 100 dernières lignes d'erreur de la DB
docker logs --tail 100 supabase-db
```

3.2 Redémarrer un service sans downtime

Si le Storage ou Realtime plante, ne redémarre pas TOUT Supabase. Redémarre juste le service concerné :

```
cd /home/bawa/supabase/docker
docker compose restart storage
```

3.3 Nettoyage de printemps (Espace Disque)

Avec les déploiements GitHub Actions (GHCR), ton serveur va accumuler des anciennes images Docker. Quand le disque est plein (Vérifier avec `df -h`), lance :

```
# Nettoie les images inutilisées, les conteneurs stoppés et les réseaux orphelins.
# (Sans danger pour les conteneurs en cours d'exécution)
docker system prune -a --volumes
```

4. Surveillance des Ressources (RAM & Disque)

4.1 Diagnostic RAM (OOM Killer)

Ce VPS dispose de 7.8 Go de RAM, ce qui est très confortable pour Supabase. Cependant, il est toujours bon de savoir comment diagnostiquer une saturation mémoire.

Quand la RAM est saturée, un mécanisme de Linux appelé OOM Killer (Out Of Memory Killer) "tue" violemment un conteneur Docker (souvent PostgreSQL ou Realtime) pour sauver le système.

Si la plateforme crash mystérieusement, voici les réflexes :

```
# 1. Vérifier la RAM libre
free -h

# 2. Vérifier si Linux a tué un process récemment (Le juge de paix)
dmesg -T | grep -i oom
```

Si la commande `dmesg` te retourne des lignes rouges parlant de `Out of memory: Killed process...`, c'est que ton serveur étouffe. Même avec 7.8 Go, une fuite de mémoire est toujours possible. Il faudra alors identifier le conteneur fautif avec `docker stats`.

4.2 Gestion de l'espace Disque (Logs Docker)

Par défaut, Docker conserve les logs pour toujours, ce qui peut finir par saturer ton disque de 150 Go sur le long terme et faire crasher le VPS.

Pour éviter cela, nous utilisons une rotation automatique des logs dans le `docker-compose.prod.yml` (Frontend) :

```
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"
```

Ici, Docker ne conserve que les 3 derniers fichiers de 10 Mo. Dès qu'on dépasse 30 Mo, le plus vieux fichier est effacé.

Si tu souhaites augmenter la taille de l'historique (par exemple, pour garder plus de traces et analyser un vieux bug) :

- Ouvre le fichier concerné (ex: `deploy/docker-compose.prod.yml`).
- Modifie `max-file: "3"` en `max-file: "10"` (tu auras alors 100 Mo d'historique disponible avec `docker logs`).
- Applique le changement en relançant le conteneur :
`docker compose -f deploy/docker-compose.prod.yml up -d`

5. Gestion des versions Frontend (Rollback Manuel)

Ton pipeline GitHub Actions inclut déjà un Rollback Automatique (si le déploiement échoue, l'ancienne version est restaurée toute seule).

Cependant, si le déploiement a réussi techniquement mais que tu te rends compte le lendemain qu'il y a un bug visuel majeur, tu vas devoir forcer un rollback manuellement depuis le VPS vers une ancienne version.

Voici comment faire étape par étape pour un débutant :

Étape 1 : Trouver le SHA

Trouve le SHA (les 7 caractères du commit) de l'ancienne version stable (regarde sur GitHub).
Exemple : `e0e0029`.

Étape 2 : Se connecter au VPS et se placer dans le bon dossier

```
ssh bawa@31.207.37.96
cd ~/termitiere-platform
```

Étape 3 : Télécharger l'ancienne image

```
docker pull ghcr.io/la-termitiere/termitiere-platform:e0e0029
```

Étape 4 : Dire à Docker Compose d'utiliser cette image

Le fichier `docker-compose.prod.yml` lit le fichier `.env.deploy` pour connaître le tag à utiliser. Il faut remplacer la ligne `IMAGE_TAG=...` existante par ton nouveau SHA sans effacer le reste du fichier (qui contient tes noms de domaine).

Ouvre le fichier avec l'éditeur :

```
nano deploy/.env.deploy
```

Va tout à la fin du fichier, supprime l'ancienne ligne `IMAGE_TAG=...` et ajoute la tienne :

```
IMAGE_TAG=e0e0029
```

Sauvegarde (`Ctrl+X` , `Y` , `Entrée`).

Étape 5 : Appliquer le retour en arrière

```
docker compose \
  --env-file deploy/.env.deploy \
  -f deploy/docker-compose.prod.yml \
  up -d --force-recreate
```

L'opération prend 5 secondes et ton site sera de retour sur l'ancienne version fonctionnelle !

6. Mises à Jour du Backend (Évolutions)

Contrairement au Frontend qui est déployé via CI/CD, le Backend (Supabase Self-Hosted) requiert une approche plus chirurgicale. Une évolution du backend concerne 3 domaines distincts :

6.1 Évolution du Schéma (Migrations SQL)

Si les développeurs ajoutent une nouvelle table ou modifient les règles de sécurité (RLS), ils produiront un script SQL (ex: `002_add_factures.sql`).

Pour l'appliquer en production, la méthode la plus robuste (sans interface web exposée) est d'injecter le fichier directement dans le conteneur PostgreSQL :

- Poussez le fichier `.sql` sur la branche principale de votre dépôt Git.
- Connectez-vous au VPS et mettez à jour les fichiers :

```
cd /home/bawa/termitiere-platform
git pull origin main
```

- Injectez le script dans la base de données :

```
# ⚠️ Remplace "002_add_factures.sql" par le vrai nom de ton fichier !
cat migration/supabase/002_add_factures.sql | docker exec -i supabase-db psql -U post
```

(Cette méthode garantit une traçabilité parfaite des opérations sur la BDD, car aucun clic aléatoire n'est fait sur une interface web).

6.2 Évolution de la Configuration (Auth, SMTP, Storage)

Si l'équipe décide d'activer Google Auth, d'ajouter un serveur d'envoi d'e-mails (SMTP) ou de changer les limites de taille du Storage, cela se passe dans les variables d'environnement de l'infrastructure Docker.

- Allez dans le dossier de configuration de Supabase :

```
cd /home/bawa/supabase/docker
```

- Modifiez le fichier `.env` (qui contient la configuration brute) :

```
nano .env
```

(Faites vos modifications, par exemple pour SMTP. Puis sauvegardez en faisant `Ctrl+X` , puis `Y` , puis `Entrée`).

- Relancez l'infrastructure Supabase pour appliquer les nouveaux réglages :

```
docker compose down
docker compose up -d
```

6.3 Évolution de la Logique Métier (Scripts Node.js)

Aujourd'hui, la logique métier côté serveur de La Termitière est gérée via des scripts Node.js (comme `import.mjs` ou `auth-migrate.mjs`). Ce n'est pas du code qui tourne en permanence, c'est du code qu'on exécute à la demande pour des opérations précises (migration de données, synchronisation, etc.).

Procédure pour déployer un nouveau script :

- Le développeur crée son script (ex: `sync-projets.mjs`) et le pousse sur la branche `main` .
- Sur le VPS, récupérez la nouvelle version du code :

```
cd /home/bawa/termitiere-platform
git pull origin main
```

- Lancez le script en suivant la Section 2.1 de ce document (installation des dépendances, variables d'environnement, exécution avec Node).

💡 Note sur les Edge Functions :

Le conteneur `supabase-edge-functions` est actif et opérationnel sur ce VPS (visible dans `docker compose ps`). Cela signifie que l'infrastructure est prête.

Cependant, aucune Edge Function personnalisée n'a encore été développée pour La Termitière. C'est une fonctionnalité qui permet de faire tourner du code TypeScript côté serveur (ex: envoyer un email automatique, générer un PDF à la demande).

Si l'équipe décide d'en créer, cela nécessitera un document dédié car c'est un sujet à part entière.